

Programowanie w języku Python

Tematy projektów zaliczeniowych

Rok akademicki 2025/2026

Zasady ogólne

Projekt zaliczeniowy polega na samodzielnym wykonaniu wybranych zadań programistycznych. Każdy student wybiera od 1 do 3 zadań z poniższej listy. Za każde poprawnie wykonane zadanie przyznawana jest określona liczba punktów. Łączna suma punktów nie może przekroczyć 100, nawet jeśli suma punktów wybranych zadań jest wyższa.

Wybór i liczba zadań

- Student wybiera maksymalnie 3 zadania.
- Każde zadanie może być wybrane tylko raz.
- Suma punktów jest ograniczona do 100 — nadwyżka ponad 100 jest odrzucana.
- Aby zaliczyć przedmiot, student musi uzyskać minimum 50 punktów.
- Temat i dziedzinę zadań wymagających zatwierdzenia (oznaczonych w treści) student konsultuje z prowadzącym najpóźniej 2 tygodnie przed oddaniem.

Przegląd zadań i punktacja

Zadanie	Tytuł	Poziom	Punkty	Maks. ocena
1	Quiz wiedzy z pliku	Łatwe	20	5,0
2	Konwerter jednostek	Łatwe	20	5,0
3	Tablica trygonometryczna	Łatwe	20	5,0
4	Dziennik ocen studenta	Średnie	30	5,0
5	Generator i analizator danych	Średnie	30	5,0
6	Baza kolekcji z pickle	Średnie	30	5,0
7	Kalkulator z historią (GUI)	Średnie	30	5,0
8	Aplikacja okienkowa z bazą danych	Trudne	40	5,0
9	System CRUD w SQLite + GUI	Trudne	40	5,0
10	Mini-aplikacja analityczna	Trudne	40	5,0

Skala ocen

Suma punktów	Ocena
0 – 49	2,0 (ndst)
50 – 59	3,0 (dst)

60 – 69	3,5 (dst+)
70 – 79	4,0 (db)
80 – 89	4,5 (db+)
90 – 100	5,0 (bdb)

Wymagania obowiązkowe dla wszystkich zadań

- Kod musi być w całości napisany przez studenta. Korzystanie z narzędzi AI jest dopuszczalne jako pomoc (wyszukiwanie, wyjaśnienia), ale student musi w pełni rozumieć każdą linię oddanego kodu.
- Projekt oddawany jest jako wydruk papierowy (w formie sprawozdania: wybrany temat, podpisane oświadczenie i samodzielnej realizacji projektu, wydruki plików programu z ewentualnymi notatkami) wraz z archiwum ZIP zawierające: wszystkie pliki .py, pliki danych (CSV, TXT, DAT), plik README.txt z instrukcją uruchomienia (należy przynieść na ostatnie zajęcia np. na nośniku typu pendrive).
- Kod musi uruchamiać się bez błędów w środowisku Python 3.10+.
- Projekt nie może importować zewnętrznych bibliotek innych niż: standardowa biblioteka Pythona, numpy, matplotlib, pandas — chyba że prowadzący wyraźnie dopuści inne.

Obrona projektu

Każde zadanie podlega krótkiej obronie ustnej (3–5 minut na zadanie) na ostatnich zajęciach zaliczeniowych. Prowadzący może:

- poprosić o uruchomienie programu na żywo z jego danymi wejściowymi,
- zadać pytanie o działanie dowolnego fragmentu kodu,
- poprosić o wprowadzenie niewielkiej modyfikacji w kodzie na żywo (np. dodanie nowej obsługi wyjątku, zmiana parametru),
- zapytać o alternatywne podejście do wybranego problemu.

Nieemożność wyjaśnienia własnego kodu lub brak reakcji na prostą prośbę modyfikacji skutkuje obniżeniem oceny lub niezaliczeniem zadania, niezależnie od jakości oddanego kodu. Adnotacje dotyczące przebiegu obrony zostaną naniesione na wydruku sprawozdania.

Zadania

Poniżej opisane są wszystkie dostępne zadania. Dla każdego podano: opis, wymagania funkcjonalne, wymagania strukturalne oraz szczegółowe kryteria oceny z punktacją.

Zadania łatwe (20 punktów)

Zadanie 1: Quiz wiedzy z pliku

20 pkt
• ŁATWE

Napisz interaktywny quiz, który wczytuje pytania z pliku tekstowego i przeprowadza użytkownika przez rozgrywkę w konsoli.

Wymagania funkcjonalne:

- Plik z pytaniami ma własny format definiowany przez studenta (np. każda linia to: pytanie|odpA|odpB|odpC|poprawna). Plik musi zawierać minimum 15 pytań z dowolnej wybranej dziedziny.
- Program wczytuje pytania z pliku, losuje ich kolejność i zadaje je użytkownikowi po kolei.

- Po każdym pytaniu wyświetlana jest informacja o poprawności odpowiedzi.
- Na końcu quizu wyświetlane jest podsumowanie: liczba punktów, procent poprawnych odpowiedzi.
- Wynik quizu (data, nick gracza, wynik) jest dopisywany do pliku historii wyników.
- Program obsługuje wyjątki: brak pliku, zły format danych w pliku, błędne dane wejściowe użytkownika.

Wymagania strukturalne:

- Minimum 3 funkcje z docstringami (tekstami pomocy) (np. wczytaj_pytania(), zadaj_pytanie(), zapisz_wynik()).
- Własny moduł utils.py lub narzedzia.py z przynajmniej jedną funkcją pomocniczą (np. formatowanie wyniku).
- Plik README.txt z opisem formatu pliku pytań i instrukcją uruchomienia.

Kryteria oceny (20 pkt):

Kryterium	Pkt
Poprawne wczytywanie i parsowanie pliku z pytaniami	4
Logika quizu: losowanie, wyświetlanie, ocenianie odpowiedzi	4
Zapis historii wyników do pliku	3
Obsługa wyjątków (min. 3 przypadki)	3
Struktura kodu: funkcje z docstringami, własny moduł	3
Jakość i czytelność kodu, README.txt	3

Zadanie 2: Konwerter jednostek fizycznych

20 pkt
• ŁATWE

Napisz program konwertujący wartości między jednostkami fizycznymi. Program działa w trybie tekstowego menu (CLI).

Wymagania funkcjonalne:

- Program obsługuje minimum 4 kategorie jednostek, np.: długość (m, km, cm, mm, cal, stopa), masa (kg, g, mg, t, funt), temperatura (°C, °F, K), czas (s, min, h, dzień).
- Dane o jednostkach i przelicznikach przechowywane są w słowniku lub wczytywane z pliku CSV — nie na sztywno w kodzie funkcji konwertującej.
- Użytkownik wybiera kategorię, jednostkę wyjściową, jednostkę docelową i podaje wartość.
- Program obsługuje wyjątki: niedozwolone wartości (np. temperatura < 0 K), błędny typ danych.
- Historia konwersji przechowywana w liście i możliwa do wyświetlenia w trakcie działania programu.

Wymagania strukturalne:

- Każda kategoria to osobna funkcja konwertująca z docstringiem.
- Własny moduł konwerter.py zawierający słowniki z jednostkami i funkcje konwersji.
- Główny plik main.py obsługuje wyłącznie menu i interakcję z użytkownikiem.

Kryteria oceny (20 pkt):

Kryterium	Pkt
Poprawność przeliczeń we wszystkich kategoriach	5
Dane jednostek w strukturze słownikowej / pliku (nie na sztywno)	4
Menu CLI i historia konwersji	3
Obsługa wyjątków i walidacja danych wejściowych	4

Zadanie 3: Tablica matematyczna z eksportem do CSV**20 pkt**
• ŁATWE

Napisz program generujący tablicę wartości wybranej funkcji matematycznej i zapisujący wyniki do pliku CSV.

Wymagania funkcjonalne:

- Program generuje dane dla minimum 3 różnych funkcji do wyboru przez użytkownika, np.: tablica trygonometryczna (sin, cos, tan, cot dla kątów 0° – 360° co $0,5^\circ$), tablica wartości funkcji potęgowej i logarytmicznej, tablica kinematyczna (droga, prędkość, przyspieszenie dla równomiernie zmiennego ruchu).
- Użytkownik wybiera funkcję, zakres i krok generowania danych.
- Wyniki zapisywane są do pliku CSV z nagłówkami kolumn.
- Program wyświetla podgląd pierwszych i ostatnich 5 wierszy wygenerowanej tablicy.
- Program obsługuje wyjątki: np. tangens w 90° , logarytm z liczby niedodatniej, dzielenie przez zero.

Wymagania strukturalne:

- Każda funkcja matematyczna to osobna funkcja Pythona z docstringiem (tekstem pomocy).
- Własny moduł `generatory.py` z funkcjami generującymi dane.
- Użycie modułów `math` lub `numpy` — bez korzystania z gotowych funkcji bibliotecznych do obliczeń, które student ma zaimplementować samodzielnie (np. `cot = cos/sin`).

Kryteria oceny (20 pkt):

Kryterium	Pkt
Poprawność obliczeń matematycznych	5
Zapis do CSV z nagłówkami, podgląd wyników	4
Wybór funkcji i parametrów przez użytkownika	3
Obsługa wyjątków matematycznych	4
Podział na moduły, docstringi, czytelność kodu	4

Zadania średnie (30 punktów)**Zadanie 4: Dziennik ocen studenta****30 pkt**
• ŚREDNIE

Napisz aplikację CLI do zarządzania ocenami studenta. Dane przechowywane są w plikach CSV. Program korzysta z własnego modułu statystycznego.

Wymagania funkcjonalne:

- Program umożliwia: dodanie oceny (przedmiot, ocena, waga, typ: kartkówka/kolokwium/egzamin/projekt, data), usunięcie oceny po ID, wyświetlenie wszystkich ocen z filtrowaniem po przedmiocie lub typie, obliczenie i wyświetlenie statystyk.
- Statystyki obejmują: średnią zwykłą, średnią ważoną, medianę, ocenę minimalną i maksymalną — dla całości i dla wybranego przedmiotu.
- Dane przechowywane w pliku CSV z nagłówkami. Przy każdym uruchomieniu program wczytuje istniejące dane.
- Program posiada tekstowe menu: D – dodaj, U – usuń, W – wyświetl, S – statystyki, Q – wyjście.

- Obsługa wyjątków: brak pliku przy pierwszym uruchomieniu, zła wartość oceny (spoza skali 2,0–5,0), zły format daty.

Wymagania strukturalne:

- Własny moduł statystyki.py z funkcjami: srednia(), srednia_wazona(), mediana(), minimum(), maksimum(). Zakazane jest użycie funkcji statistics.mean itp. — student implementuje algorytmy samodzielnie.
- Minimum 4 funkcje w głównym pliku z docstringami (tekstami pomocy).
- Podział: main.py (menu i interakcja), statystyki.py (obliczenia), plik_csv.py lub analogiczny (operacje na pliku).

Kryteria oceny (30 pkt):

Kryterium	Pkt
Poprawne operacje CRUD na pliku CSV	6
Moduł statystyki.py z poprawnymi algorytmami (bez bibliotek statystycznych)	7
Menu CLI, filtrowanie, wyświetlanie wyników	5
Obsługa wyjątków i walidacja danych	5
Architektura wieloplikowa, docstringi	4
Czytelność kodu i plik README.txt	3

Zadanie 5: Generator i analizator danych pomiarowych

30 pkt
• ŚREDNIE

Napisz program, który generuje dane symulujące pomiary rzeczywistego zjawiska, zapisuje je do CSV, a następnie wczytuje i analizuje statystycznie.

Wymagania funkcjonalne:

- Student wybiera własną dziedzinę zjawiska (np. wzrost i masa osób, tętno w trakcie treningu, temperatury dobowe, ceny nieruchomości, wyniki pomiarów laboratoryjnych). Dane muszą być fizycznie sensowne i uzasadnione w README.
- Generator tworzy minimum 500 rekordów z wykorzystaniem rozkładu normalnego (Gausa). Każdy rekord zawiera minimum 4 pola, w tym przynajmniej dwie zmienne numeryczne i jedno pole kategoryczne.
- Dane zapisywane są do pliku CSV.
- Moduł analizujący wczytuje plik CSV i oblicza dla każdej zmiennej numerycznej: min, max, średnią, medianę, odchylenie standardowe.
- Wyniki analizy wyświetlane są w czytelnej tabeli tekstowej w konsoli i zapisywane do pliku raport.txt.

Wymagania strukturalne:

- Moduł generator.py — generowanie i zapis danych.
- Moduł statystyki.py — wszystkie funkcje statystyczne zaimplementowane samodzielnie (bez numpy.std itp.).
- Moduł analizator.py — wczytanie CSV, wywołanie statystyk, formatowanie i zapis raportu.
- README.txt z uzasadnieniem wyboru dziedziny i przyjętych parametrów rozkładu.

Kryteria oceny (30 pkt):

Kryterium	Pkt
Poprawny generator danych z rozkładem normalnym, sensowne parametry	6
Własne funkcje statystyczne (bez gotowych bibliotek statystycznych)	8
Wczytywanie CSV i generowanie raportu tekstowego	5

Architektura 3-modułowa	5
Obsługa wyjątków, walidacja, docstringi	4
README z uzasadnieniem i jakością kodu	2

Zadanie 6: Baza kolekcji z pickle i menu CLI

30 pkt
• ŚREDNIE

Napisz aplikację do zarządzania dowolną kolekcją obiektów. Dane przechowywane są za pomocą modułu pickle. Aplikacja posiada rozbudowane menu tekstowe.

Wymagania funkcjonalne:

- Student definiuje własną dziedzinę kolekcji: filmy, gry, książki, albumy muzyczne, karty kolekcjonerskie, postacie z gry RPG lub inne — zatwierdzone z prowadzącym.
- Każdy obiekt posiada minimum 5 atrybutów, w tym przynajmniej: jeden tekstowy, jedna lista (np. tagi, zdolności, autorzy), jeden numeryczny.
- Program umożliwia: dodawanie nowego obiektu, usuwanie po nazwie/ID, wyszukiwanie po dowolnym atrybucie, wyświetlanie całej kolekcji, zapis i odczyt z pliku .dat (pickle).
- Menu główne: D – dodaj, U – usuń, S – szukaj, W – wyświetl wszystkie, Q – zapisz i wyjdź.
- Przy każdym uruchomieniu program automatycznie wczytuje istniejący plik .dat (jeżeli istnieje).

Wymagania strukturalne:

- Klasa reprezentująca obiekt kolekcji z metodą `__str__()` i `__repr__()`. Wszystkie atrybuty inicjowane w `__init__`.
- Plik `kolekcja.py` z klasą, plik `operacje.py` z funkcjami CRUD, plik `main.py` z menu.
- Minimum 3 metody w klasie poza `__init__` i `__str__` (np. `edytuj_atrybut()`, `porownaj()`, `jako_slownik()`).
- W kolekcji musi znajdować się minimum 10 obiektów wprowadzonych przez studenta (nie generowanych losowo).

Kryteria oceny (30 pkt):

Kryterium	Pkt
Klasa z minimum 5 atrybutami i metodami (OOP)	8
Poprawny zapis i odczyt z pickle, automatyczne wczytywanie	6
Operacje CRUD: dodawanie, usuwanie, wyszukiwanie	6
Menu CLI i obsługa wyjątków	5
Architektura 3-plikowa, docstringi	3
Własnoręcznie wprowadzone dane (min. 10 obiektów)	2

Zadanie 7: Kalkulator z historią i interfejsem okienkowym

30 pkt
• ŚREDNIE

Napisz kalkulator naukowy z graficznym interfejsem użytkownika (tkinter), który przechowuje historię obliczeń w pliku.

Wymagania funkcjonalne:

- Kalkulator obsługuje: cztery działania arytmetyczne, potęgowanie i pierwiastkowanie, minimum 3 funkcje matematyczne (np. `sin`, `cos`, `log`, silnia, wartość bezwzględna).
- Interfejs GUI (tkinter): pole wyświetlacza, przyciski cyfr i działań, przycisk C (wyczyść) i CE (usuń ostatni znak).

- Historia obliczeń widoczna w osobnym panelu (np. Listbox lub ScrolledText) — wyświetla ostatnie 20 operacji.
- Historia jest zapisywana do pliku historia.txt i wczytywana przy każdym uruchomieniu programu.
- Obsługa błędów: dzielenie przez zero, pierwiastek z liczby ujemnej, log z liczby ≤ 0 — komunikat w GUI, nie crash programu.

Wymagania strukturalne:

- Plik kalkulator.py z funkcjami obliczeniowymi (logika biznesowa) — bez tkinter.
- Plik gui.py lub main.py z kodem interfejsu — wywołuje funkcje z kalkulator.py.
- Każda funkcja obliczeniowa posiada docstring z przykładem użycia.

Kryteria oceny (30 pkt):

Kryterium	Pkt
Poprawność obliczeń, wszystkie wymagane operacje	6
GUI tkinter: czytelny układ, działające przyciski	8
Historia: wyświetlanie w GUI, zapis/odczyt z pliku	6
Obsługa błędów w GUI (komunikat, brak crasha)	5
Podział logika/GUI, docstringi	5

Zadania trudne (40 punktów)

Zadanie 8: Aplikacja okienkowa z bazą danych SQLite

40 pkt

• TRUDNE

Napisz pełną aplikację okienkową (tkinter) zarządzającą bazą danych SQLite. Aplikacja realizuje kompletny cykl CRUD dla danych z wybranej dziedziny.

Wymagania funkcjonalne:

- Student wybiera dziedzinę aplikacji: biblioteka domowa, baza filmów, magazyn produktów, dziennik treningów, lista zadań projektowych lub inna — zatwierdzona z prowadzącym.
- Baza SQLite zawiera minimum 2 tabele połączone relacją (klucz obcy). Przykład: tabela Autorzy + tabela Książki.
- Aplikacja realizuje wszystkie operacje CRUD: dodaj, edytuj, usuń, szukaj (z filtrowaniem po co najmniej 2 polach).
- Wyniki wyświetlane są w kontrolce Treeview (tabela w GUI).
- Minimum jedna operacja JOIN — wyświetlanie danych z połączonych tabel w Treeview.
- Walidacja danych w GUI przed wysłaniem do bazy — komunikat o błędzie wyświetlany w GUI.

Wymagania strukturalne:

- Plik baza.py — wyłącznie operacje SQL (tworzenie tabel, INSERT, SELECT, UPDATE, DELETE). Żadnego kodu GUI.
- Plik gui.py — wyłącznie kod tkinter. Wywołuje funkcje z baza.py.
- Plik main.py — punkt wejścia aplikacji.
- Schemat bazy (diagram lub opis tabel z polami i relacjami) w pliku schemat.txt.

Kryteria oceny (40 pkt):

Kryterium	Pkt
Schemat bazy: 2 tabele z relacją, poprawne typy danych	7
Kompletne operacje CRUD działające przez GUI	10

Treeview z danymi, operacja JOIN widoczna w widoku	8
Walidacja danych i komunikaty błędów w GUI	6
Separacja kodu: baza.py / gui.py / main.py	5
Docstringi, schemat.txt, README.txt	4

Zadanie 9: System zarządzania danymi: SQLite + GUI + eksport

40 pkt
• TRUDNE

Napisz rozbudowaną aplikację łączącą bazę SQLite, interfejs tkinter i eksport danych do CSV. Aplikacja obsługuje dane z minimum 3 powiązanych tabel.

Wymagania funkcjonalne:

- Dziedzina do wyboru przez studenta (np. system rejestracji studentów na kursy, ewidencja sprzętu w firmie, system rezerwacji, sklep internetowy uproszczony) — zatwierdzona z prowadzącym.
- Baza SQLite zawiera minimum 3 tabele z relacjami (klucze obce). Minimum jedno zapytanie z JOIN obejmującym wszystkie 3 tabele.
- GUI tkinter: zakładki (Notebook) dla różnych sekcji aplikacji (np. Studenci, Kursy, Zapisy).
- Kompletny CRUD dla każdej z tabel, dostępny z poziomu GUI.
- Funkcja eksportu: wybrany widok danych (wynik zapytania SELECT) eksportowany do pliku CSV.
- Funkcja wyszukiwania z dynamicznym filtrowaniem — wyniki aktualizują się po wpisaniu frazy.

Wymagania strukturalne:

- Architektura MVC: model.py (SQL), widok.py lub gui.py (tkinter), kontroler.py (logika łącząca oba).
- Plik inicjalizujący bazę init_db.py — tworzy tabele i wstawia przykładowe dane startowe.
- Minimum 20 rekordów danych testowych wprowadzonych przez studenta (nie generowanych losowo).
- Schemat bazy w pliku schemat.txt z opisem tabel, pól i relacji.

Kryteria oceny (40 pkt):

Kryterium	Pkt
Schemat bazy: 3 tabele z relacjami, poprawność SQL	7
Kompletny CRUD dla wszystkich tabel przez GUI	9
GUI z zakładkami, Treeview, dynamiczne wyszukiwanie	8
Eksport do CSV, min. 20 rekordów danych testowych	6
Architektura MVC, init_db.py, schemat.txt	7
Docstringi i jakość kodu	3

Zadanie 10: Mini-aplikacja analityczna z wizualizacją

40 pkt
• TRUDNE

Napisz aplikację, która wczytuje dane z pliku CSV, przeprowadza ich analizę statystyczną i generuje zestaw wykresów. Aplikacja posiada prosty interfejs GUI lub zaawansowane CLI z raportowaniem.

Wymagania funkcjonalne:

- Dane wejściowe: student samodzielnie pozyskuje lub generuje zestaw danych CSV zawierający minimum 200 rekordów i minimum 5 kolumn (min. 3 numeryczne, min. 1 kategorię). Dziedzina dowolna — dane muszą być rzeczywiste lub realistycznie symulowane z uzasadnieniem.
- Analiza statystyczna dla każdej zmiennej numerycznej: min, max, średnia, mediana, odchylenie standardowe, współczynnik zmienności — wszystkie funkcje zaimplementowane samodzielnie.

- Wizualizacja (matplotlib): minimum 4 różne typy wykresów, np. histogram rozkładu, wykres pudełkowy (boxplot), wykres rozproszenia (scatter), wykres słupkowy dla zmiennej kategorycznej.
- Wszystkie wykresy zapisywane do plików PNG, dodatkowo generowany jest zbiorczy raport w pliku raport.txt.
- Interfejs: GUI tkinter z możliwością wyboru kolumny do analizy i generowania wykresów na żądanie — LUB zaawansowane CLI z argumentami wywołania (moduł argparse).

Wymagania strukturalne:

- Moduł statystyki.py — własne funkcje statystyczne (bez numpy.mean, statistics.mean itp.).
- Moduł wykresy.py — wszystkie funkcje generujące wykresy (matplotlib).
- Moduł wczytaj_dane.py — wczytywanie i walidacja CSV.
- Plik main.py lub gui.py — punkt wejścia z interfejsem.
- README.txt z opisem danych: źródło lub opis symulacji, liczba rekordów, opis kolumn.

Kryteria oceny (40 pkt):

Kryterium	Pkt
Dane: 200+ rekordów, 5+ kolumn, realistyczne i opisane	6
Własne funkcje statystyczne dla wszystkich miar	9
4 typy wykresów, zapis do PNG, estetyka wykresów	9
Raport tekstowy raport.txt z wynikami analizy	5
Interfejs GUI lub argparse CLI	6
Architektura 4-modułowa, docstringi, README.txt	5

Powodzenia!

Pytania dotyczące wyboru tematów lub wymagań kieruj do prowadzącego w trakcie zajęć lub drogą mailową.